

the speed to deploy containers, with little overhead and very consistent behaviours between the various stages of development and production.

Benefits of containers	Limitations of containers
No need for a full OS in each instance, reducing memory and CPU usage.	Share the host OS, offering less isolation than VMs.
Launch in seconds, speeding up development and deployment cycles.	Limited by the host OS (e.g., Linux containers on Linux).
Run consistently across development, testing, and production environments.	Improper setup can expose vulnerabilities or host access.
Ideal for microservices and large-scale deployments with tools like Kubernetes.	Managing storage across container lifecycles can be complex.
More containers can run on the same hardware than VMs.	Container networking can require additional configuration and monitoring.

Table 2: Benefits and limitations of containers

Virtual machines are suitable for running legacy apps that have full operating system requirements and strict compatibility concerns. VMs can also be useful when you need to run various operating systems on a single physical machine — several VMs can run on and consume the resources of a single physical machine. This strong isolation provided by VMs is useful for software where security is critical or is not designed to run in a containerised world.

Often, in enterprise environments, a hybrid design offers the best results. For example, running containers inside VMs can afford organisations the same security and OS-level governance they usually require in a VM while benefiting from the advantages of containers with respect to portability and speed.

Common misconceptions about containers and VMs

As containers continue to gain popularity, a few misconceptions have also taken root. First, it’s important to understand that containers are not just lightweight versions of virtual machines. While they serve similar purposes in isolating applications, the way they operate is fundamentally different. Containers share the host OS and are designed for fast, scalable workloads, whereas VMs offer full OS-level isolation.

Aspect	 Containers	 Virtual Machines
Architecture	Containers share the host operating system and package only the app and its dependencies.	Virtual machines run a full guest operating system on top of a hypervisor.
Performance	They are lightweight, use fewer resources, and start almost instantly.	VMs use more resources and take longer to boot because they run a full OS.
Portability	Containers move easily between environments as long as the host OS is compatible.	VMs can run on different hardware and operating systems, offering broader portability.
Security	They share the host OS kernel, which means they have weaker isolation and a larger attack surface.	VMs offer stronger isolation with separate operating systems, making them more secure.
Management	Containers are managed with tools like Docker and orchestrators such as Kubernetes.	VMs are managed with hypervisor platforms like VMware vSphere.
Scalability	Containers are built for quick horizontal scaling and work great for microservices.	VMs can scale but usually come with more overhead and are often used for legacy applications.

Table 3: Key differences between VMs and containers